PC40 Lab 3 – Final Synthesis Note

Bora Yalçın

1. Language and machine

Toolchain C++17 (`-O2 -std=c++17 -Wall -Wextra -pedantic`, `-pthread` for the parallel files), g++ 15.2.0 (Ubuntu). `double` precision throughout.

Machine Intel Core Ultra 9 285H, 16 cores, no SMT, hybrid: 6 P-cores (5.4 GHz) + 8 E-cores (4.5 GHz) + 2 LP-E cores (2.5 GHz). Kubuntu 26.04, kernel 7.0.0-38.

Data MNIST and the supplied `-synthetic` generator.

Network 28×28×1 → conv 3×3/8 filters → 26×26×8 → ReLU → pool 2×2 → 13×13×8 → flatten 1352 → dense 10 → softmax; 13 610 trainable parameters.

2. Decompositions and ownership rule

v1 — image ownership Worker t owns images $[\lfloor tB/T \rfloor, \lfloor (t+1)B/T \rfloor)$ (Eq. 2) and runs the whole forward pass for them in its own one-image `WS`. One join, no sync between layers, since the same thread does conv→pool→dense for its own images.

v2 — row ownership Worker t owns a chunk of a flat (n, f, r) row index. Three phases (conv rows → pool rows → dense+softmax) over batch-wide buffers (≈51 MB at B=500), separated by 2 barriers (C++17 `Barrier`, Appendix A), then join.

Training (Part C) Image ownership again. Each worker owns a private full-network gradient G_t (13 610 doubles), zeroes it, and runs the reference `forward+loss+backward_accumulate` (which only *adds* into G_t, reading θ_τ only); no worker calls `sgd_update`. After join, the main thread reduces $G = \sum_{t=0}^{T-1} G_t$ in increasing worker order (same for the loss) and applies exactly one update $\theta_{\tau+1} = \theta_\tau - (\eta/B)G$, dividing once by the actual batch size B.

Ownership rule Every output location has exactly one writer. Everything else (images, θ_τ) is read-only or private to one worker during the computation, so the only writes that can conflict are pushed into the single reduction, done by one thread.

3. Correctness evidence

Sequential reference `-self-test` PASS (hand convolution error 0, forward vs. naive 2.22×10^{-16}, 0 wrong pooling winners); `-gradcheck` PASS, max error 2.26×10^{-10} against central finite differences.

Parallel vs. reference `validate_D.cpp` compares the reference against v1/v2/train, each in its own namespace: **104 comparisons, 0 failures**.

Forward Values (S, A, P, win, z, p, inference checksum) are **exact** (max error 0) for 1 image, uneven partitions (B=5, T=3), $T > B$, a pooling tie, B=37, T=5, and MNIST.

Gradients At fixed θ_τ (tolerance $10^{-10} + 10^{-8} \max(|a|, |b|)$, Eq. 8): **max abs error** 3.78×10^{-15}, max rel error 2.13×10^{-12}, over every uneven case tested, including B=32, T=7 (chunks 4+5+4+5+4+5+5).

Updates One parameter update: max diff 1.39×10^{-17}. A 3-epoch run (189 updates): losses within 1.11×10^{-15}, params within 5.55×10^{-16}, for $T = 1, 3, 16$.

Why not bit-identical For $T > 1$ only because the reduction order changes (floating-point addition is not associative) – never because of a bug.

4. Compact KPI table

T	Infer t_T (s)	S_T	E_T	throughput (img/s)	Train b32 S_T	Train b256 S_T
1	1.1433	1.00	1.00	43 733	1.00	1.00
2	0.6432	1.78	0.89	77 736	1.59	1.75
4	0.4106	2.78	0.70	121 762	2.01	2.74
8	0.4459	2.56	0.32	112 142	1.65	2.91
16	0.3205^{pool}	2.84	0.18	156 023	1.39	**3.14**

Table 1: Inference B=500, v1 (create mode except T=16, which uses the pool: 0.4869 s create / 0.3205 s pool); sequential 0.9153 s. Extra memory: v1 0.11–1.65 MB, v2 flat 51.46 MB. Training: sequential b32 0.4292 s, b256 0.4272 s, image ownership, create per batch, 10 000 MNIST images.

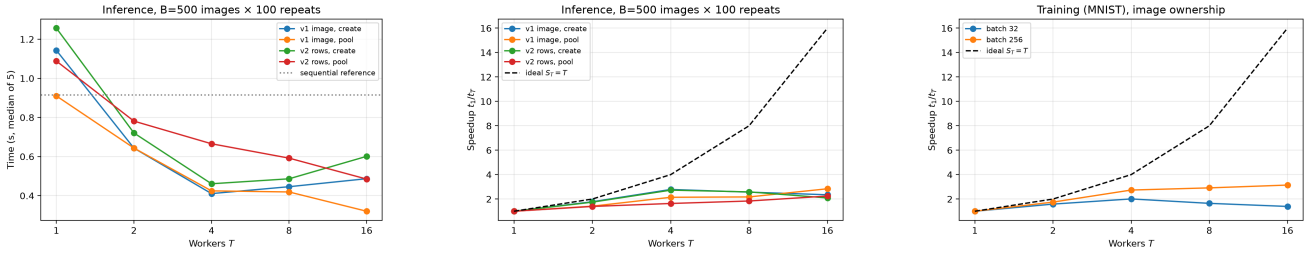


Figure 1: Left/mid: inference B=500 time and speedup vs. T (v1/v2, create/pool). Right: training speedup vs. T, batch 32 vs. batch 256 – same code, 8× more work per sync point, better scaling.

Two measured but unverified observations (reported as observations with candidate causes only):

(1) Create-mode T=1 is 25–40% slower than sequential, while the pool's T=1 (main thread does the work) is not. *Candidate cause:* the one created thread may land on a slower core or start with cold caches.

(2) v2 with the pool never reaches v2 create's best speedup at B=500 (pool peak S_T=2.24 vs. create's 2.73). *Candidate cause:* v2's bottleneck is its 2 barriers and 51 MB buffers, not thread creation, so the pool (which only removes thread creation) helps it less than it helps v1.

5. Short answers

Where is synchronization necessary?
At every point one worker's write must become visible to a different worker before it reads:
- the join after a batch of images (v1, training);
- the two barriers between conv→pool→dense phases when workers share one batch's row-owned buffers (v2);
- the single `sgd_update` after the reduction, so no worker ever reads a half-updated θ.

Why do some shared reads need no lock?
A data race needs at least one write. The input images, the parameters θ during a forward pass, and the batch during training are read by every worker but written by none of them while they are being read – many concurrent unchanged reads cannot race.

Why is gradient reduction required?
The same filter is reused at every spatial position, so G_K, G_b are a sum over positions and over images. Each worker can only sum its own images locally, so a second, explicit sum over the T workers is needed before one update – exactly like Fig. 9's lost-update example if that sum were done by an unsynchronized shared accumulator instead.

What limits the measured speedup?
- thread creation per call/batch (create vs. pool: 0.49 vs. 0.32 s at T=16, inference);
- a serial reduction that grows with T (0.0068→0.0867 s, batch 32, training);
- too little work per task at B=1<T (sync costs more than an 18 µs image) and at batch 32, T=16 (2 images/worker);
- unequal P/E/LP-E cores, so a join waits for the slowest one.

The effective serial fraction fitted from the measured speedup therefore grows with T (0.26→0.70, batch 32) instead of staying constant as Amdahl's model assumes, because these costs grow with T too.

6. Conclusion

What worked Bit-identical forward in both decompositions, validated training (gradients $\leq 3.8 \times 10^{-15}$), 2.86× inference and 2.55× training speedup vs. sequential.

What did not scale B=1 < T (sync costs more than the 18 µs of work) and batch-32 training past T≈4 (2 images/worker, reduction grows with T).

What I would improve next A persistent pool for training too, a lighter/spinning barrier, a parallel or tree reduction, larger work per task, and dynamic scheduling for the hybrid cores.